

1. Cloud Service Provider and AWS Services

The cloud service provider selected for the Online Course Registration System is **Amazon Web Services (AWS)**. AWS was selected because it provides scalable cloud infrastructure, reliable database services, and security tools that are appropriate for a web-based course registration application. AWS is also a good choice for this project because it supports the services needed for application hosting, database deployment, networking, and access control.

AWS is cost-effective for a student project because the system can begin with small resources and scale later if more users or features are added. For example, the application can start with a small EC2 instance and a managed RDS database. If the number of users increases, the team can upgrade the server size or add additional resources without redesigning the entire system.

The main AWS services used in this architecture are:

AWS Service	Purpose
Amazon EC2	Hosts the web/application server for the course registration system.
Amazon RDS for MySQL	Stores course, student, instructor, section, and enrollment data.
Amazon VPC	Provides a secure private network for the application resources.
Public Subnets	Host internet-facing resources such as the application server.
Private Subnets	Host protected resources such as the database.
Security Groups	Control inbound and outbound traffic to the application and database.
Amazon S3	Can store static files, backups, or supporting project documents if needed.

2. Operating System and Virtual Servers

The application server will run on a Linux-based operating system, such as **Ubuntu Server** or **Amazon Linux**. Linux was selected because it is commonly used for web application hosting, works well with Node.js and Express, and does not require additional Windows licensing costs. Linux is also stable and widely supported in cloud environments.

For the initial version of the system, the application can run on a small EC2 instance. This is appropriate because the project is a prototype and does not need a large production server at the start.

Server Component	Selected Configuration
Virtual Server	Amazon EC2
Operating System	Ubuntu Server or Amazon Linux
Instance Type	t3.micro or similar small instance
CPU	2 vCPUs
Memory	1 GB RAM
Storage	20 GB SSD
Runtime Support	Node.js and Express application environment

This configuration is enough for development, testing, and a small course registration prototype. If the system later needs to support more users, the EC2 instance can be resized or the architecture can be expanded to include a load balancer and additional application servers.

3. Network Architecture and Security

The system will use an **Amazon Virtual Private Cloud (VPC)** to organize and protect cloud resources. The VPC will include both public and private subnets. Public-facing resources, such as the web/application server, will be placed in a public subnet. The database will be placed in a private subnet so it cannot be accessed directly from the internet.

To support resiliency and high availability, the architecture should include subnets across at least two Availability Zones. This means the design can continue to support the application if one Availability Zone has an issue. A simple version of this design includes two public subnets and two private subnets across two Availability Zones.

The application server will receive user traffic from the internet. Users will access the system through a browser using HTTP or HTTPS. The application server will then communicate with the RDS MySQL database in the private subnet. The database will only accept traffic from the application server security group.

Security Group and Port Configuration

Resource	Allowed Inbound Traffic	Purpose
Application Server Security Group	Port 80 from internet	Allows HTTP web traffic.
Application Server Security Group	Port 443 from internet	Allows HTTPS secure web traffic.
Application Server Security Group	Port 22 from approved admin IPs only	Allows limited SSH administrative access.
Database Security Group	Port 3306 from application server only	Allows MySQL communication between the application and database.

Network Architecture Diagram

User Browser / Internet

|

| HTTPS 443 / HTTP 80

v

AWS VPC

| |

| Availability Zone 1 |

| Public Subnet |

| - EC2 Application Server |

| - Security Group: allows 80/443, limited 22 |

| |

| Private Subnet |

| - RDS MySQL Database |

| - Security Group: allows 3306 from EC2 only |

| |

| Availability Zone 2 |

| Public Subnet for future app redundancy |

| Private Subnet for database standby/backup |

| |

Database response returns through the EC2 application server.

This diagram shows that users access the application server through the public subnet, while the RDS MySQL database remains protected in a private subnet and only accepts traffic from the application server.

Only the ports required for the application should be open. The database should not allow public internet access. SSH access should be limited to approved project team or administrator IP addresses. This reduces the risk of unauthorized access while still allowing the application to operate.

This network design separates the public application layer from the private data layer. It supports security because users never connect directly to the database. It also supports future scalability because the system can later add more application servers, load balancing, or stronger monitoring if needed.

Application Design

Programming Language

The online course registration system will be developed using Javascript. Javascript was selected because it is widely used for web application development and works well with the [Node.js](#) runtime environment. Using Javascript allows the application to be developed efficiently while supporting communication between the client interface and the backend services.

Runtime Environment

The application will run on the [node.js](#) runtime environment hosted on the Amazon EC2 instance. Node.js is scalable, lightweight, and is capable of handling multiple client requests. It can also integrate well with express and supports the RESTful API architecture used by the application.

Application API

The system will use a Restful API to allow communication between the user interface and application server. The API will use standard HTTP methods using GET, POST, PUT, and DELETE. Data will be exchanged using JSON, providing a simple and consistent format for communication from frontend to backend.

HTTP Method	Endpoint	Purpose
GET	/courses	Retrieve all available courses
GET	/courses/:id	Retrieve information about a specific course
POST	/students	Register a new student

POST	/login	Authenticate a user
GET	/students/:id	Retrieve student profile information
PUT	/students/:id	Update student information
POST	/enrollments	Register a student for a course
DELETE	/enrollments/:id	Remove a course enrollment
GET	/enrollments	View all enrollments

Application framework

The application's presentation layer will be developed using React. React provides a component-based architecture that allows developers to build reusable user interface components while delivering a responsive and interactive user experience. React also communicates efficiently with REST APIs, making it an ideal choice for this application.

Middleware

The application will use Express.js as the middleware framework. Express simplifies request routing, API development, middleware integration, and communication between the frontend and the MySQL database. It also provides support for handling authentication, validation, and error management.

System architecture Layers

The application follows a traditional three-tier architecture consisting of the presentation layer, application layer, and data layer.

Presentation Layer

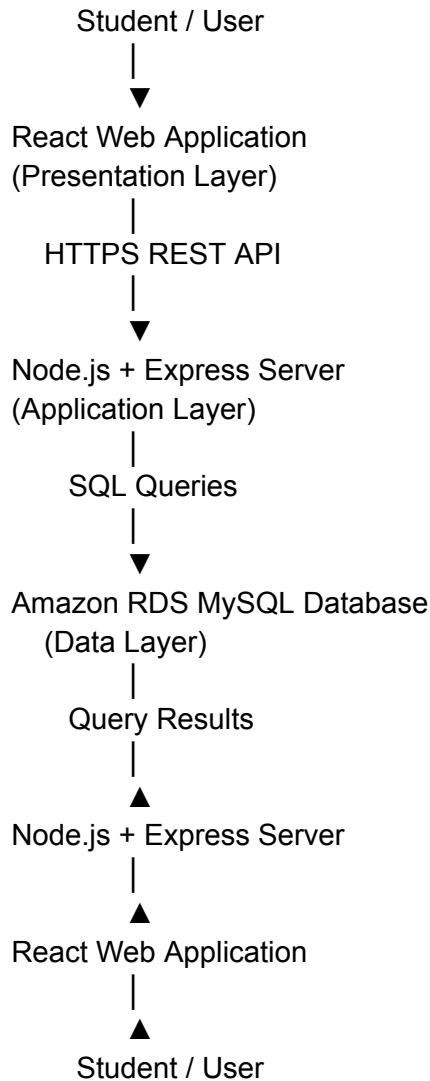
- React web application
- Student, Instructor, and Administrator interfaces
- Displays information and accepts user input

Application Layer

- Node.js runtime
- Express.js framework
- Processes business logic
- Validates requests
- Communicates with the database

Data Layer

- Amazon RDS MySQL database
- Stores student, instructor, course, section, and enrollment records
- Returns requested data to the application layer



4. Database Design and Normalization

Relational Database Selection

The core data repository for the Online Course Registration System is deployed on an Amazon RDS for MySQL engine. A relational database management system (RDBMS) was selected due to the highly structured, transactional nature of academic records, which requires strict data integrity, operational consistency, and robust constraint enforcement.

Data Normalization (3rd Normal Form)

To eliminate data redundancy and prevent operational anomalies (insertion, update, and deletion anomalies), the underlying database storage schema has been systematically designed to adhere strictly to **Third Normal Form (3NF)**:

- **First Normal Form (1NF):** All tables enforce atomic cell entries (no multi-valued attributes) and distinct, unique rows established by primary keys.
- **Second Normal Form (2NF):** The schema meets 1NF criteria and eliminates partial dependencies. Every non-prime attribute depends completely on the entire primary key, which is maintained across composite intersections using surrogate auto-increment identifiers.
- **Third Normal Form (3NF):** The schema meets 2NF criteria and completely eliminates transitive dependencies. Non-key columns depend solely on the primary key, and nothing but the primary key. Attributes like course properties, department mappings, and instructor attributes are isolated into standalone functional domains, connected exclusively via foreign key constraints.

Database Physical Schema Reference

The database instance compiles six distinct tables organized to handle academic logic natively:

Table Name	Primary Key (PK)	Foreign Key(s) (FK)	Core Non-Key Attributes
departments	department_id	<i>None</i>	department_name
instructors	instructor_id	department_id	first_name, last_name, email
courses	course_id	department_id	course_code, course_title, credits
sections	section_id	course_id, instructor_id	semester, year, room_number
students	student_id	<i>None</i>	first_name, last_name, email

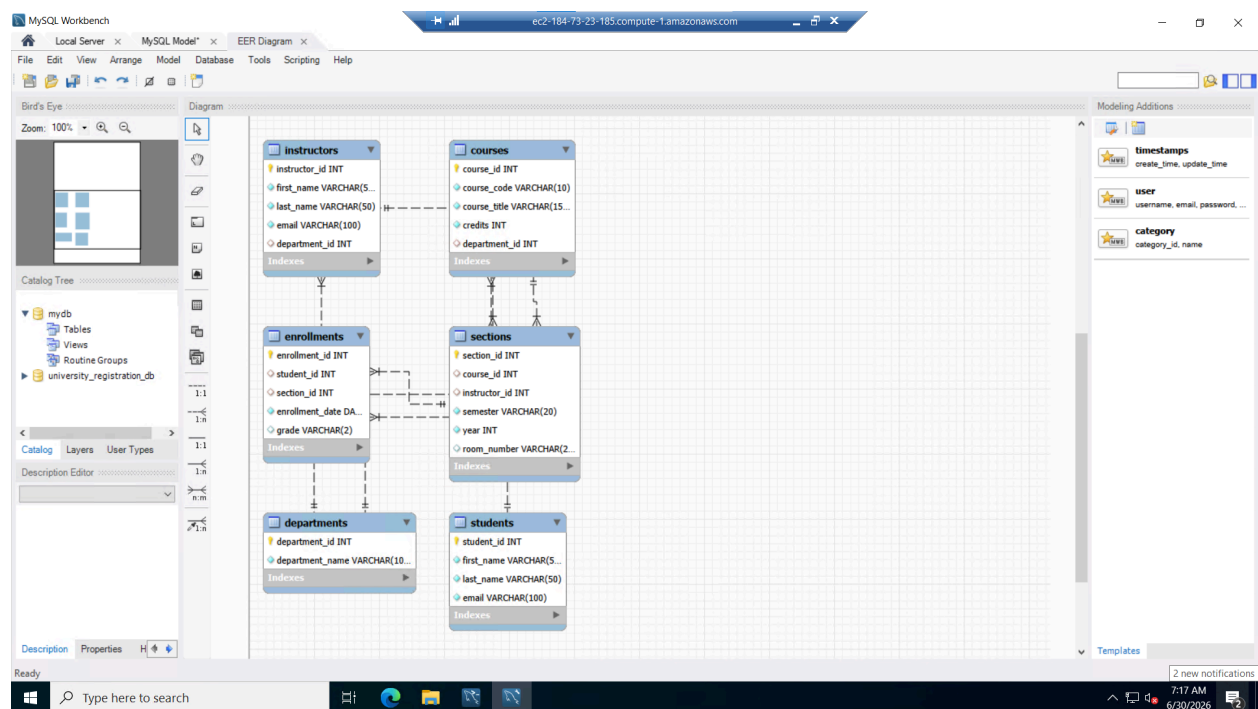
enrollments	enrollment_id	student_id, section_id	enrollment_date, grade
--------------------	---------------	---------------------------	------------------------

Database Implementation and Verification Artifacts

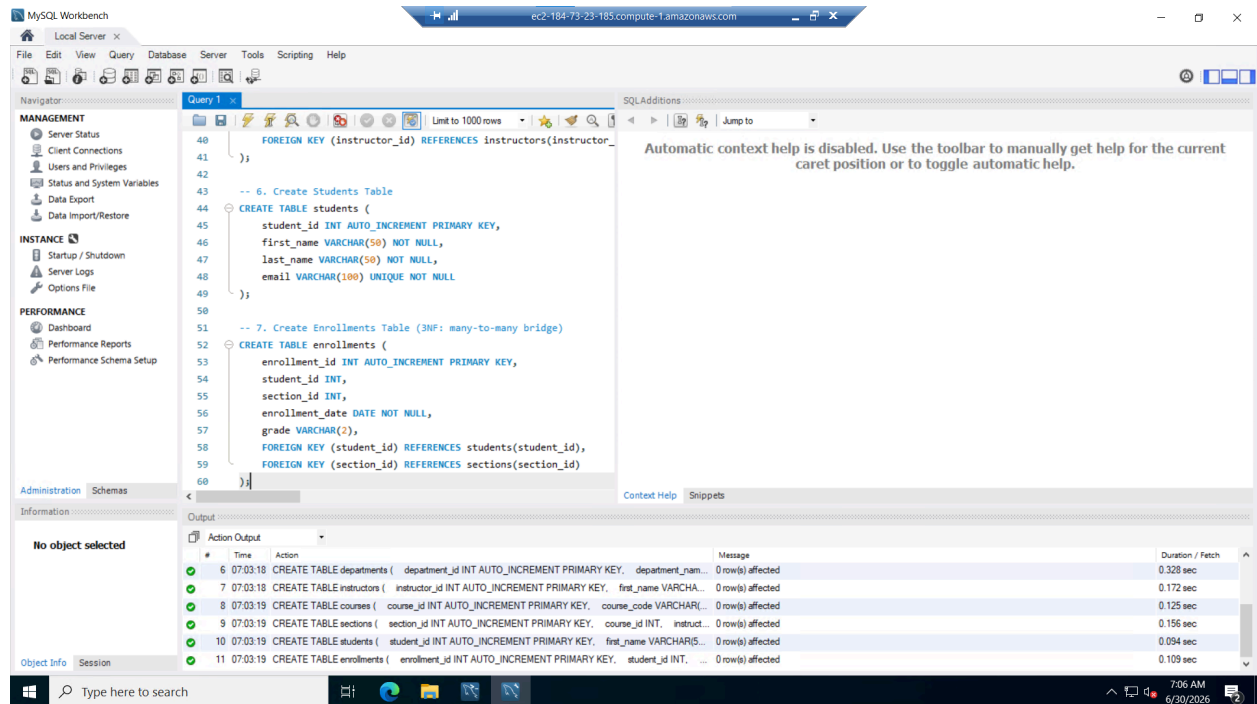
The DDL (Data Definition Language) architecture script was successfully loaded and executed on the virtualized target environment. This verified the logical syntax and structural layout of our third normal form relations within the operational backend system.

Entity Relationship Diagram (ERD) Model

Below is the structural visualization of the physical database layout showing how primary and foreign keys interface to support systemic relationships:



Below is the execution verification showing the creation of the normalized structure in the running server cluster:



Testing and Quality Assurance Process

To ensure the Online Course Registration System operates correctly and reliably, the project will follow a testing strategy that includes unit testing, integration testing, and end-to-end testing. These testing activities will be performed throughout development to identify defects early and verify that the application functions as expected.

Unit Testing

Unit testing will verify that individual application components function correctly in isolation. The development team will test core application logic such as course retrieval, student registration, enrollment validation, and input validation. Each API endpoint and business logic function will be tested independently to ensure it produces the expected results before being integrated with other system components.

Integration Testing

Integration testing will verify communication between the React frontend, the Node.js and Express application server, and the Amazon RDS MySQL database. Testing will confirm that API requests are processed correctly, database queries return the expected data, and information is transferred accurately between all application layers. This testing ensures that independently developed components work together as a complete system.

End-to-End Testing

End-to-end testing will validate the complete user experience from the browser to the database and back to the user. Common user workflows, such as logging in, viewing available courses, registering for a course, updating student information, and dropping a course, will be executed to verify that the application behaves as expected. End-to-end testing confirms that the entire system functions correctly under normal operating conditions.

6. Authentication and Authorization Process

The Online Course Registration System will support multiple user roles to ensure users can access only the features appropriate to their responsibilities. Authentication and authorization implementation are outside the scope of this architecture assignment; however, the expected system roles are defined below.

Student

Students can log into the system to view available courses, register for course sections, view current enrollments, update their personal information, and drop enrolled courses. Students are permitted to access only their own academic records and account information.

Instructor

Instructors can view the courses and sections assigned to them, access class rosters, and manage student grades where applicable. Instructors cannot modify student accounts or system-wide administrative settings.

Administrator

Administrators have full access to the system. Administrative responsibilities include managing students, instructors, departments, courses, sections, and enrollment records. Administrators are also responsible for maintaining system configuration, managing user accounts, and performing overall application administration.

These roles provide a role-based access model that limits user permissions according to their responsibilities while protecting sensitive academic and administrative data.

Team Responsibilities

To ensure all architecture requirements were completed efficiently, responsibilities were divided among team members based on the major sections of the project.

Dylan – Cloud & Network/Security Architect

Dylan was responsible for designing the cloud infrastructure and network architecture for the application. His responsibilities included:

- Selecting **Amazon Web Services (AWS)** as the cloud service provider and providing justification.
- Configuring AWS services including **EC2, RDS, VPC, subnets, and security groups**.
- Designing the virtual network architecture with public and private subnets for high availability.
- Selecting the operating system and virtual server configuration.
- Defining firewall rules, security groups, ingress/egress rules, and required application ports.
- Ensuring the infrastructure met security, scalability, and availability requirements.

Marlon – Application Developer

Marlon was responsible for the application architecture and system design. His responsibilities included:

- Selecting the programming language, runtime environment, framework, and middleware.
- Designing the REST API and documenting the core application endpoints.
- Defining the presentation, application, and data layers.
- Creating the complete request flow showing how a user request moves through the application and database.
- Developing the overall system architecture diagram illustrating communication between the client, application server, and database.

Claytis – Database Architect

Claytis was responsible for designing and documenting the database architecture. His responsibilities included:

- Creating the Entity-Relationship (ER) diagram.
- Selecting the database management system (**MySQL on AWS RDS**).
- Designing database tables, relationships, primary keys, and foreign keys.
- Normalizing the database to **Third Normal Form (3NF)**.
- Ensuring the database design supports data integrity, scalability, and efficient querying.

Team Collaboration

All team members collaborated throughout the project by reviewing each other's work, providing feedback, ensuring consistency across all architecture components, and maintaining the shared GitHub

repository. Regular communication and peer review helped ensure the final architecture document was complete, technically accurate, and met all assignment requirements before submission.

Git hub URL: https://github.com/KrispyClay/UI-UX_Assignment.git